

Common Table Expressions (CTEs) in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [Theory: What is a CTE?](#)
 3. [Simple CTE](#)
 4. [Multiple CTEs](#)
 5. [CTE vs Subquery vs Derived Table](#)
 6. [Recursive CTEs](#)
 7. [Hierarchical Data with Recursive CTEs](#)
 8. [CTE MATERIALIZED and NOT MATERIALIZED](#)
 9. [CTEs with DML \(INSERT, UPDATE, DELETE\)](#)
 10. [ASCII Visual Diagrams](#)
 11. [Common Mistakes](#)
 12. [Best Practices](#)
 13. [Performance Considerations](#)
 14. [Interview Questions & Answers](#)
 15. [Hands-on Exercises](#)
 16. [Advanced Notes](#)
 17. [Cross-references](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Write simple and multiple CTEs for query decomposition
 - Understand the difference between CTE and derived table behavior
 - Write recursive CTEs to traverse hierarchical data
 - Control CTE materialization in PostgreSQL 12+
 - Use CTEs inside DML statements
-

Theory: What is a CTE?

A Common Table Expression (CTE), written with the WITH clause, is a named temporary result set that exists only for the duration of one query. It can be referenced in the main query (and other CTEs in the same WITH block).

Key Properties

1. Defined with WITH name AS (SELECT ...)
 2. Visible to the main query and subsequent CTEs in the same WITH block
 3. Cannot be referenced after the query ends
 4. In PostgreSQL 12+, CTEs are INLINED by default (not materialization barriers)
 5. Can be RECURSIVE (self-referencing) for tree/graph traversal
 6. Can be used in SELECT, INSERT, UPDATE, DELETE
-

Simple CTE

Syntax

```
WITH cte_name AS (  
    SELECT ...  
)  
SELECT ...  
FROM cte_name  
[WHERE ...];
```

Examples

```
-- Example 1: Simple CTE to improve readability  
WITH high_earners AS (  
    SELECT emp_id, emp_name, department, salary  
    FROM emp_salary  
    WHERE salary > 80000  
)  
SELECT  
    department,  
    COUNT(*) AS count,  
    AVG(salary) AS avg_salary  
FROM high_earners  
GROUP BY department  
ORDER BY avg_salary DESC;  
  
-- Example 2: CTE to avoid repeating a complex expression  
WITH monthly_stats AS (  
    SELECT  
        department,  
        month_date,  
        revenue,  
        AVG(revenue) OVER (PARTITION BY department) AS avg_rev,  
        MAX(revenue) OVER (PARTITION BY department) AS max_rev  
    FROM monthly_revenue  
)  
SELECT  
    department,  
    month_date,  
    revenue,  
    ROUND(revenue / avg_rev * 100, 2) AS pct_of_avg,  
    CASE WHEN revenue = max_rev THEN 'Peak Month' ELSE '' END AS is_peak  
FROM monthly_stats  
ORDER BY department, month_date;  
  
-- Example 3: CTE for filtering before aggregation  
WITH recent_sales AS (  
    SELECT *
```

```

    FROM sales
    WHERE sale_date >= CURRENT_DATE - INTERVAL '90 days'
),
rep_summary AS (
    SELECT
        rep_name,
        COUNT(*) AS sale_count,
        SUM(amount) AS total_rev
    FROM recent_sales
    GROUP BY rep_name
)
SELECT *
FROM rep_summary
WHERE total_rev > 1000
ORDER BY total_rev DESC;

```

Multiple CTEs

Multiple CTEs can be chained — each can reference CTEs defined before it.

```

-- Example 4: Multi-CTE query – step-by-step analysis
WITH
-- Step 1: Compute per-rep totals
rep_totals AS (
    SELECT rep_name, region, SUM(amount) AS revenue
    FROM sales
    WHERE region IS NOT NULL
    GROUP BY rep_name, region
),
-- Step 2: Rank reps within each region
rep_ranked AS (
    SELECT
        rep_name,
        region,
        revenue,
        RANK() OVER (PARTITION BY region ORDER BY revenue DESC) AS region_rank
    FROM rep_totals
),
-- Step 3: Get only top-2 per region
top_reps AS (
    SELECT * FROM rep_ranked WHERE region_rank <= 2
)
-- Final: join with original data for context
SELECT t.region, t.rep_name, t.revenue, t.region_rank
FROM top_reps t
ORDER BY t.region, t.region_rank;

-- Example 5: CTE pipeline – data cleaning and reporting
WITH
raw_data AS (

```

```

SELECT *
FROM sales
WHERE amount > 0          -- remove negative/zero entries
      AND sale_date IS NOT NULL -- exclude bad records
),
normalized AS (
  SELECT
    rep_name,
    COALESCE(region, 'Unknown') AS region,
    product,
    sale_date,
    amount,
    units_sold
  FROM raw_data
),
aggregated AS (
  SELECT
    region,
    product,
    SUM(amount)      AS revenue,
    SUM(units_sold) AS total_units,
    COUNT(*)         AS num_sales
  FROM normalized
  GROUP BY region, product
)
SELECT
  region,
  product,
  revenue,
  total_units,
  ROUND(revenue / NULLIF(total_units, 0), 2) AS rev_per_unit
FROM aggregated
ORDER BY revenue DESC;

```

CTE vs Subquery vs Derived Table

Feature	CTE	Subquery	Derived Table
Named reference	YES	NO	YES (alias)
Reusable in query	YES	NO	NO
Can be recursive	YES	NO	NO
Readable	HIGH	MEDIUM	LOW (nested)
Performance (PG12+)	Inlined	Inlined	Inlined
Can use with DML	YES	YES	NO

```
-- Same query three ways:
```

```

-- Derived table approach:
SELECT region, AVG(rep_rev)
FROM (SELECT region, rep_name, SUM(amount) AS rep_rev
      FROM sales GROUP BY region, rep_name) AS rep_totals
GROUP BY region;

-- Subquery approach:
SELECT region, (SELECT AVG(sub.rep_rev)
                FROM (SELECT rep_name, SUM(amount) AS rep_rev
                      FROM sales s2 WHERE s2.region = s.region
                      GROUP BY rep_name) sub) AS avg_rep_rev
FROM (SELECT DISTINCT region FROM sales WHERE region IS NOT NULL) s;

-- CTE approach (most readable):
WITH rep_totals AS (
  SELECT region, rep_name, SUM(amount) AS rep_rev
  FROM sales WHERE region IS NOT NULL
  GROUP BY region, rep_name
)
SELECT region, AVG(rep_rev) AS avg_rep_rev
FROM rep_totals
GROUP BY region;

```

Recursive CTEs

Recursive CTEs reference themselves, enabling traversal of hierarchical or graph data.

Syntax

```

WITH RECURSIVE cte_name AS (
  -- Anchor member (non-recursive base case)
  SELECT ...

  UNION [ALL]

  -- Recursive member (references cte_name)
  SELECT ...
  FROM cte_name
  JOIN ...
  WHERE termination_condition
)
SELECT * FROM cte_name;

```

Execution Model

```

Step 1: Execute anchor member → seed result set (R0)
Step 2: Execute recursive member using R0 as input → R1
Step 3: Execute recursive member using R1 as input → R2
...

```

Step N: Recursive member produces empty result → STOP
Final: UNION ALL of R0, R1, R2, ..., RN-1

Examples

```
-- Example 6: Count from 1 to 10 (simplest recursive CTE)
WITH RECURSIVE counter(n) AS (
    SELECT 1                                -- anchor
    UNION ALL
    SELECT n + 1 FROM counter              -- recursive
    WHERE n < 10                           -- termination
)
SELECT n FROM counter;

-- Example 7: Date sequence generator
WITH RECURSIVE date_series(d) AS (
    SELECT '2024-01-01'::DATE
    UNION ALL
    SELECT d + INTERVAL '1 day'
    FROM date_series
    WHERE d < '2024-01-31'::DATE
)
SELECT d AS date FROM date_series;

-- Example 8: Fibonacci sequence
WITH RECURSIVE fib(n, a, b) AS (
    SELECT 1, 0, 1
    UNION ALL
    SELECT n + 1, b, a + b
    FROM fib
    WHERE n < 15
)
SELECT n, a AS fibonacci FROM fib;

-- Example 9: Employee hierarchy traversal
WITH RECURSIVE org_chart AS (
    -- Anchor: top-level employees (no manager)
    SELECT
        emp_id,
        emp_name,
        manager_id,
        department,
        salary,
        0 AS depth,
        emp_name::TEXT AS path
    FROM emp_salary
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive: employees whose manager is already in the CTE
    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id,
        e.department,
        e.salary,
        m.depth + 1 AS depth,
        m.path || e.emp_name AS path
    FROM emp_salary e
    JOIN org_chart m ON e.manager_id = m.emp_id
```

```

SELECT
    e.emp_id,
    e.emp_name,
    e.manager_id,
    e.department,
    e.salary,
    oc.depth + 1,
    oc.path || ' → ' || e.emp_name
FROM emp_salary e
JOIN org_chart oc ON e.manager_id = oc.emp_id
)
SELECT
    REPEAT(' ', depth) || emp_name AS indented_name,
    department,
    salary,
    depth,
    path
FROM org_chart
ORDER BY path;

```

Hierarchical Data with Recursive CTEs

```

-- Example 10: Find all descendants of a given employee
WITH RECURSIVE subordinates AS (
    SELECT emp_id, emp_name, manager_id, 0 AS level
    FROM emp_salary
    WHERE emp_id = 1    -- start from Alice (CEO)

    UNION ALL

    SELECT e.emp_id, e.emp_name, e.manager_id, s.level + 1
    FROM emp_salary e
    JOIN subordinates s ON e.manager_id = s.emp_id
)
SELECT
    REPEAT(' ', level) || emp_name AS hierarchy,
    level,
    emp_id
FROM subordinates
ORDER BY level, emp_name;

-- Example 11: Upward traversal – find all ancestors of an employee
WITH RECURSIVE ancestors AS (
    SELECT emp_id, emp_name, manager_id, 0 AS level
    FROM emp_salary
    WHERE emp_id = 6    -- start from Paul (intern)

    UNION ALL

    SELECT e.emp_id, e.emp_name, e.manager_id, a.level + 1

```

```

        FROM emp_salary e
        JOIN ancestors a ON e.emp_id = a.manager_id
    )
    SELECT emp_name, level, emp_id
    FROM ancestors
    ORDER BY level;

-- Example 12: Compute depth of hierarchy levels
WITH RECURSIVE depths AS (
    SELECT emp_id, emp_name, manager_id, 1 AS depth
    FROM emp_salary WHERE manager_id IS NULL

    UNION ALL

    SELECT e.emp_id, e.emp_name, e.manager_id, d.depth + 1
    FROM emp_salary e
    JOIN depths d ON e.manager_id = d.emp_id
)
SELECT department, MAX(depth) AS max_depth, COUNT(*) AS emp_count
FROM depths
JOIN emp_salary USING (emp_id)
GROUP BY department;

```

CTE MATERIALIZED and NOT MATERIALIZED

In PostgreSQL 12+, CTEs are inlined (NOT MATERIALIZED) by default, meaning the optimizer can push predicates through them. Use MATERIALIZED to force the CTE to be computed once and stored.

```

-- Example 13: Force materialization (pre-compute once, reuse)
WITH MATERIALIZED expensive_computation AS (
    SELECT
        department,
        SUM(salary) AS dept_total,
        COUNT(*) AS headcount,
        AVG(salary) AS avg_sal
    FROM emp_salary
    GROUP BY department
)
SELECT e.emp_name, e.salary, ec.dept_total, ec.avg_sal
FROM emp_salary e
JOIN expensive_computation ec ON e.department = ec.department;

-- Example 14: NOT MATERIALIZED (default in PG12+) – allow planner to inline
WITH NOT MATERIALIZED recent_orders AS (
    SELECT * FROM orders WHERE order_date >= '2024-01-01'
)
SELECT c.customer_name, o.total_amount
FROM customers c
JOIN recent_orders o ON c.customer_id = o.customer_id;

-- Planner may push down additional WHERE conditions into the CTE

```

CTEs with DML

CTEs can be used with INSERT, UPDATE, and DELETE.

```
-- Example 15: CTE with UPDATE – give raises to low earners
WITH low_earners AS (
    SELECT emp_id
    FROM emp_salary
    WHERE salary < (SELECT AVG(salary) * 0.8 FROM emp_salary)
)
UPDATE emp_salary
SET salary = salary * 1.15
WHERE emp_id IN (SELECT emp_id FROM low_earners);

-- Example 16: Data archival – move old orders to archive table
WITH deleted_orders AS (
    DELETE FROM orders
    WHERE order_date < '2023-01-01'
    RETURNING *
)
INSERT INTO orders_archive
SELECT * FROM deleted_orders;

-- Example 17: INSERT with CTE
WITH new_customer AS (
    INSERT INTO customers (customer_name, city, country)
    VALUES ('Zara Ling', 'Seattle', 'USA')
    RETURNING customer_id
)
INSERT INTO orders (customer_id, order_date, total_amount)
SELECT customer_id, CURRENT_DATE, 0
FROM new_customer;
```

ASCII Visual Diagrams

CTE Execution Flow

```
WITH
  cte_a AS (SELECT ...),           ← evaluated lazily (or eagerly if MATERIALIZED)
  cte_b AS (SELECT ... FROM cte_a) ← can reference cte_a
SELECT * FROM cte_b WHERE ...;    ← main query references both
```

Execution order (logical):

cte_a → cte_b → main query filter → result

Recursive CTE Execution

```
WITH RECURSIVE hierarchy AS (
  [ANCHOR] → Level 0: CEO rows
```

```

↓
[RECURSIVE] Level 0 as input → Level 1: Direct reports
↓
[RECURSIVE] Level 1 as input → Level 2: Indirect reports
↓
[RECURSIVE] Level 2 as input → empty set
↓ STOP
UNION ALL: Level 0 + Level 1 + Level 2

```

Tree produced:

```

CEO (depth=0)
├── Alice (depth=1)
│   └── Paul (depth=2)
├── Mary (depth=1)
└── Tom (depth=1)

```

Common Mistakes

Mistake 1: Infinite recursion — missing termination condition

```

-- WRONG: no WHERE clause to stop recursion
WITH RECURSIVE r AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM r    -- no LIMIT or WHERE!
)
SELECT * FROM r; -- runs forever (or until recursion limit)

-- CORRECT:
WITH RECURSIVE r AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM r WHERE n < 100 -- termination condition
)
SELECT * FROM r;

```

Mistake 2: Using UNION instead of UNION ALL in recursive CTE

```

-- UNION deduplicates at each step – can prevent proper traversal
-- and is much slower. Always use UNION ALL in recursive CTEs.
-- Only use UNION if you specifically need to detect cycles via dedup.

```

Mistake 3: Expecting CTE to always be materialized (pre-PG12 behavior)

```

-- In PG12+, CTEs are inlined by default – the same CTE referenced twice
-- may be executed twice (if not materialized).
-- Use WITH MATERIALIZED to guarantee single execution.

```

```

WITH expensive AS MATERIALIZED (
    SELECT ...(expensive query)...
)
SELECT * FROM expensive e1 JOIN expensive e2 ON ...;
-- Without MATERIALIZED, PG12+ might execute the CTE twice!

```

Mistake 4: Referencing a CTE that was defined after the current CTE

```

-- CTEs are processed in order – cte_b cannot reference cte_c
WITH
    cte_b AS (SELECT * FROM cte_c), -- ERROR: cte_c not yet defined
    cte_c AS (SELECT 1)
SELECT * FROM cte_b;

```

Best Practices

1. Use CTEs to break complex queries into readable named steps.
2. Use MATERIALIZED when the CTE is expensive and referenced multiple times.
3. Prefer CTEs over deep subquery nesting for readability.
4. Always include a termination condition in recursive CTEs.
5. Use UNION ALL (not UNION) in recursive CTEs for performance.
6. Add a depth/level counter and a MAX_DEPTH guard to prevent runaway recursion.
7. Use the WITH clause for DML pipelines (delete-and-archive patterns).

Performance Considerations

```

-- PostgreSQL 12+ inlines CTEs by default – optimizer can push predicates
-- This is usually GOOD (better selectivity)

-- Force materialization when:
-- 1. CTE is expensive to compute
-- 2. CTE is referenced multiple times
-- 3. You need isolation from outer query predicates

-- Max recursion depth (default 100):
SET max_recursion_depth = 1000; -- increase for deep hierarchies

-- Recursive CTE index usage – index on the join column:
CREATE INDEX idx_emp_manager ON emp_salary(manager_id);
-- The recursive step: JOIN emp ON emp.manager_id = cte.emp_id
-- will use this index for efficient lookup

-- Explain to verify:
EXPLAIN (ANALYZE, FORMAT TEXT)
WITH RECURSIVE org AS (
    SELECT * FROM emp_salary WHERE manager_id IS NULL
    UNION ALL
    SELECT e.* FROM emp_salary e JOIN org ON e.manager_id = org.emp_id

```

```
)  
SELECT * FROM org;
```

Interview Questions & Answers

Q1: What is a CTE and how is it different from a subquery?

A: A CTE (Common Table Expression) is a named temporary result set defined with the WITH clause, scoped to a single query. Unlike an inline subquery or derived table, a CTE can be referenced by name multiple times within the same query, can be recursive, and improves readability by breaking complex queries into named logical steps.

Q2: What is a recursive CTE?

A: A recursive CTE (WITH RECURSIVE) has two parts joined by UNION ALL: an anchor member that starts the result set, and a recursive member that joins the CTE to itself to extend the result one level at a time. It continues until the recursive member produces no new rows. Used for hierarchical data (org charts, BOM, trees, graphs).

Q3: What is the difference between UNION and UNION ALL in a recursive CTE?

A: UNION ALL is almost always used in recursive CTEs because UNION performs deduplication at each recursion step, which is expensive and can cause incorrect results by pruning valid paths. Use UNION only if you explicitly need cycle detection via deduplication.

Q4: How does PostgreSQL 12+ change CTE behavior?

A: Before PostgreSQL 12, CTEs were always materialized (optimization fence — outer predicates could not be pushed inside). From PostgreSQL 12+, CTEs are NOT MATERIALIZED by default (inlined), allowing the optimizer to push predicates through them. Use MATERIALIZED explicitly to force the old behavior.

Q5: How do you prevent infinite recursion in a recursive CTE?

A: Add a WHERE condition in the recursive member that limits depth (WHERE depth < 10), or maintain a list of visited nodes to detect cycles.

Q6: Can CTEs be used in DML statements?

A: Yes. CTEs can wrap UPDATE, DELETE, and INSERT in a WITH block. A common pattern is `DELETE ... RETURNING *` inside a CTE, then `INSERT ... SELECT` from the CTE to implement atomic move/archive operations.

Q7: Can a CTE reference another CTE defined in the same WITH block?

A: Yes, but only CTEs that were defined before the referencing one. CTEs are processed in order within the WITH block.

Q8: What is the optimization fence behavior of CTEs?

A: In PostgreSQL pre-12, CTEs acted as optimization fences — the planner could not push WHERE predicates from the outer query into the CTE. This sometimes hurt performance (the CTE computed too many rows) but sometimes helped (guaranteed single evaluation). In PG12+, this fence is removed by default.

Hands-on Exercises

Exercise 1: Three-Step Analysis

Using multiple CTEs: (1) compute total revenue per rep, (2) rank reps by revenue, (3) show only top 50%.

```
-- Solution:
WITH
rep_revenue AS (
    SELECT rep_name, region, SUM(amount) AS revenue
    FROM sales WHERE region IS NOT NULL
    GROUP BY rep_name, region
),
rep_ranked AS (
    SELECT *, NTILE(2) OVER (ORDER BY revenue DESC) AS half
    FROM rep_revenue
),
top_half AS (
    SELECT * FROM rep_ranked WHERE half = 1
)
SELECT rep_name, region, revenue
FROM top_half
ORDER BY revenue DESC;
```

Exercise 2: Generate a Date Series

Use a recursive CTE to generate all dates in January 2024.

```
-- Solution:
WITH RECURSIVE jan_dates(d) AS (
    SELECT '2024-01-01'::DATE
    UNION ALL
    SELECT d + 1 FROM jan_dates WHERE d < '2024-01-31'::DATE
)
SELECT d AS date, TO_CHAR(d, 'Day') AS day_name FROM jan_dates;
```

Exercise 3: Org Chart

Display the full organizational hierarchy from the emp_salary table with indentation showing depth.

```
-- Solution:
WITH RECURSIVE org AS (
    SELECT emp_id, emp_name, manager_id, department,
           0 AS depth,
           ARRAY[emp_id] AS path
    FROM emp_salary WHERE manager_id IS NULL

    UNION ALL

    SELECT e.emp_id, e.emp_name, e.manager_id, e.department,
           o.depth + 1,
           o.path || e.emp_id
    FROM emp_salary e
    JOIN org o ON e.manager_id = o.emp_id;
```

```

        JOIN org o ON e.manager_id = o.emp_id
        WHERE NOT e.emp_id = ANY(o.path) -- cycle guard
    )
    SELECT
        REPEAT(' ', depth) || emp_name AS hierarchy,
        department,
        depth
    FROM org
    ORDER BY path;

```

Exercise 4: CTE with DELETE ... RETURNING

Archive all sales from before 2024 by moving them to a temp archive table.

```

-- Solution:
CREATE TEMP TABLE sales_archive AS SELECT * FROM sales WHERE FALSE;

WITH archived AS (
    DELETE FROM sales
    WHERE sale_date < '2024-01-01'
    RETURNING *
)
INSERT INTO sales_archive
SELECT * FROM archived;

-- Verify:
SELECT COUNT(*) FROM sales_archive;

```

Advanced Notes

Cycle Detection in Recursive CTEs

```

-- PostgreSQL 14+: native CYCLE clause
WITH RECURSIVE graph_traversal AS (
    SELECT node_id, neighbor_id, 0 AS depth
    FROM edges WHERE node_id = 1
    UNION ALL
    SELECT e.node_id, e.neighbor_id, g.depth + 1
    FROM edges e JOIN graph_traversal g ON e.node_id = g.neighbor_id
)
CYCLE node_id SET is_cycle USING cycle_path
SELECT * FROM graph_traversal WHERE NOT is_cycle;

-- Manual cycle detection using arrays (pre-PG14):
WHERE NOT new_node = ANY(visited_nodes_array)

```

SEARCH Clause (PostgreSQL 14+)

```
-- Control depth-first vs breadth-first traversal order:
```

```
WITH RECURSIVE tree AS (...)
```

```
SEARCH DEPTH FIRST BY emp_id SET ordercol
```

```
SELECT * FROM tree ORDER BY ordercol;
```

```
-- or BREADTH FIRST:
```

```
SEARCH BREADTH FIRST BY emp_id SET ordercol
```

Cross-references

- **05_recursive_queries.md** — Deep dive into trees, graphs, BOM with recursive CTEs
- **03_subqueries.md** (03_Intermediate_SQL) — CTE vs subquery comparison
- **01_window_functions_intro.md** — Window functions inside CTEs
- **08_interview_window_tasks.md** — Many problems use CTEs for staging